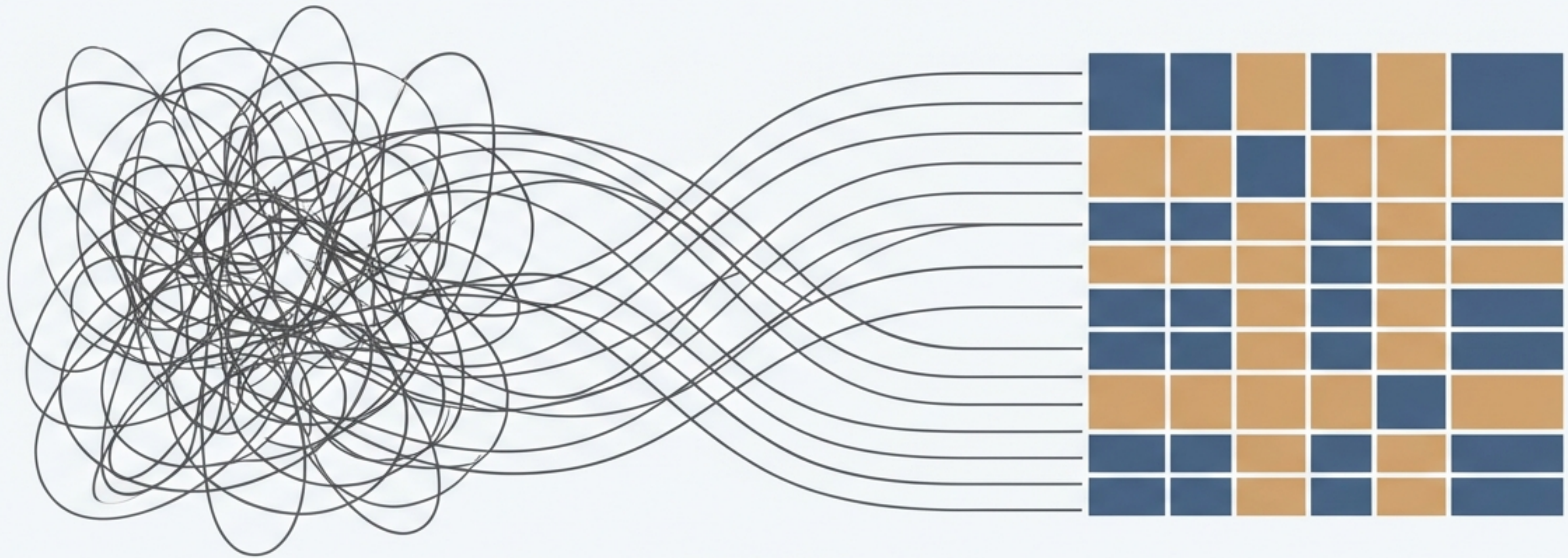


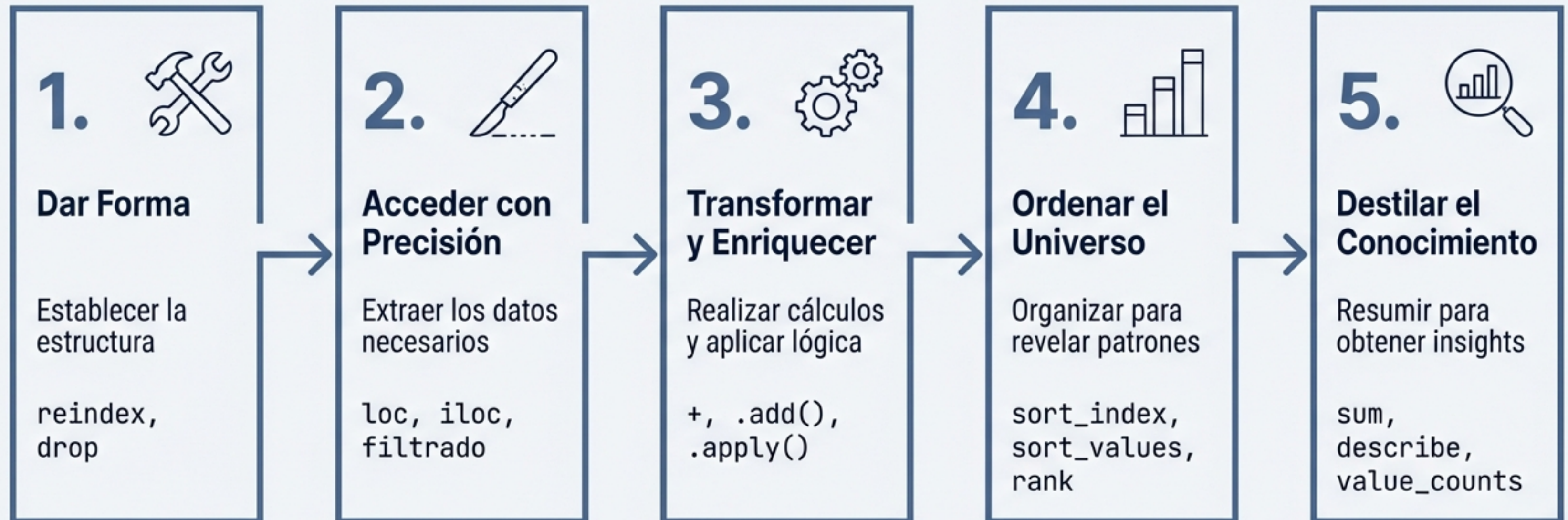
Dominando Pandas: Un Viaje por la Manipulación de Datos

De los Datos Crudos al Insight: Funcionalidades Esenciales



Nuestro Flujo de Trabajo: Del Caos a la Claridad

Todo análisis de datos es un viaje. Comenzamos con datos que necesitan ser moldeados, seleccionamos las partes que importan, las transformamos para crear nuevo valor, las organizamos para ver patrones y, finalmente, las resumimos para extraer conocimiento. Pandas proporciona las herramientas para cada paso.



Remodelando la Estructura con `reindex`

`reindex` crea un nuevo objeto con los datos conformados a un nuevo índice. Es crucial entender que las etiquetas nuevas sin datos correspondientes se rellenan con `NaN` (Not a Number).

Sección 1: Ejemplo con Series

Antes: `obj`

obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])	
d	4.5
b	7.2
a	-5.3
c	3.6
dtype: float64	

obj2 = obj.reindex(["a", "b", "c", "d", "e"])



Después: `obj2`

d	4.5
b	7.2
a	-5.3
c	3.6
e	NaN
dtype: float64	

Sección 2: Ejemplo con DataFrame

Antes: `frame`

frame = pd.DataFrame(np.arange(9).reshape(3,3), index=["a", "c", "d"], columns=["Ohio", "Texas", "California"])				
	Ohio	Texas	California	
a	0	1	2	
c	3	4	5	
d	6	7	8	

states = ["Texas", "Utah", "California"];
frame2 = frame.reindex(states, axis="columns")



Después: `frame2`

	Texas	Utah	California
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Argumentos Clave de `reindex`

`reindex` es una función versátil con múltiples argumentos para controlar su comportamiento.

Argumento	Descripción
<code>labels</code>	Nueva secuencia a utilizar como índice.
<code>index</code>	Alias para especificar el índice de filas.
<code>columns</code>	Alias para especificar el índice de columnas.
<code>axis</code>	Eje a reindexar (`index/0` por defecto, `columns/1`).
<code>method</code>	Método para rellenar valores faltantes (ej. `ffill` para rellenar hacia adelante, `bfill` hacia atrás).
<code>fill_value</code>	Valor específico para usar en lugar de `NaN` para los valores faltantes.
<code>limit</code>	Número máximo de elementos a rellenar al usar `method`.
<code>copy</code>	Controla si se crea una nueva copia de los datos subyacentes.

Refinando el Dataset: Eliminando Entradas con `drop`

El método `drop` devuelve un nuevo objeto con las etiquetas indicadas eliminadas de un eje. Es una forma segura y explícita de quitar datos.

Sección 1: Ejemplo con Series

Antes: `obj`

a	0.0
b	1.0
c	2.0
d	3.0
e	4.0
dtype: float64	

```
obj2 = obj.drop(["c","e"])
```



Después: `obj2`

a	0.0
b	1.0
d	3.0
dtype: float64	

Sección 2: Ejemplo con DataFrame

Antes: `frame`

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

```
frame2 = frame.drop(index="Ohio",  
columns=["Cuatro","Dos"])
```



Después: `frame2`

	Uno	Tres
Texas	4	6
California	8	10
Utah	12	14

Accediendo a los Datos con Precisión Quirúrgica

Pandas ofrece dos operadores principales para la selección de datos, y es vital entender su diferencia:

.loc

Por Etiqueta

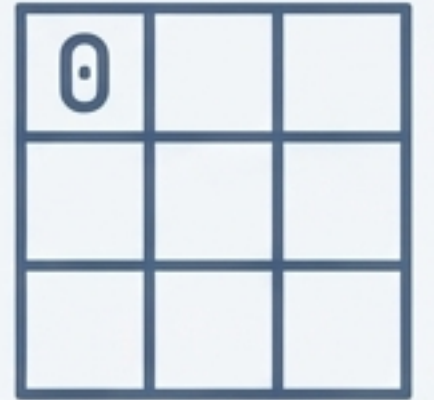


Selecciona datos basándose en los **nombres** (labels) del índice y las columnas. Es inclusivo en los rangos.

```
frame.loc["Texas", "Uno"]
```

.iloc

Por Posición



Selecciona datos basándose en la **posición** entera (empezando en 0). Es exclusivo en el límite superior de los rangos, como en Python.

```
frame.iloc[1, 0]
```

Recomendación: Aunque se puede usar '[' directamente, es una fuente común de errores. Usar '.loc' e '.iloc' es más explícito, legible y previene comportamientos inesperados.

Selección y Filtrado en Acción

Usaremos el siguiente DataFrame para todos los ejemplos en esta diapositiva.

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

Selección de Múltiples Columnas por Etiqueta

```
frame.loc[:, ["Uno", "Dos"]]
```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15



	Uno	Dos
Ohio	0	1
Texas	4	5
California	8	9
Utah	12	13

Selección de Filas y Columnas por Etiqueta

```
frame.loc["Texas", ["Uno", "Dos"]]
```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15



Texas
Uno: 4
Dos: 5
Name, dtype: int64

Selección por Posición Entera

```
frame.iloc[[1, 2], [3, 0, 1]]
```

	0	1	2	3
	Uno	Dos	Tres	Cuatro
0 Ohio	0	1	2	3
1 Texas	4	5	6	7
2 California	8	9	10	11
3 Utah	12	13	14	15



	Cuatro	Uno	Dos
Texas	7	4	5
California	11	8	9

Filtrado Booleano

```
frame[frame["Uno"] > 5]
```

Uno		Uno	Dos	Tres	Cuatro
0	Ohio	0	1	2	3
4	Texas	4	5	6	7
8	California	8	9	10	11
12	Utah	12	13	14	15



	Cuatro	Uno	Dos
California	7	4	5
Utah	11	8	9

Aritmética Inteligente: La Magia de la Alineación de Datos

Al realizar operaciones aritméticas entre objetos de Pandas, los datos se alinean automáticamente por sus etiquetas de índice y columna. Si un índice no se encuentra en ambos objetos, el resultado en esa posición es `NaN`.

El Problema: `NaN`'s Inesperados

frame1					frame2					
	a	b	c	d		a	b	c	d	e
0	0.0	1.0	2.0	3.0	0	0.0	1.0	2.0	3.0	4.0
1	4.0	5.0	6.0	7.0	1	5.0	6.0	7.0	8.0	9.0
2	8.0	9.0	10.0	11.0	2	10.0	11.0	12.0	13.0	14.0
					3	15.0	16.0	17.0	18.0	19.0

suma1 = frame1 + frame2

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	NaN
1	9.0	11.0	13.0	15.0	NaN
2	18.0	20.0	22.0	24.0	NaN
3	NaN	NaN	NaN	NaN	NaN

La Solución: Rellenar Valores con `fill\_value`

Para manejar la falta de solapamiento, podemos usar métodos aritméticos (como `.add()`) y especificar un `fill\_value`. Esto sustituye los valores faltantes *antes* de la operación.

frame1					frame2					
	a	b	c	d		a	b	c	d	e
0	0.0	1.0	2.0	3.0	0	0.0	1.0	2.0	3.0	4.0
1	4.0	5.0	6.0	7.0	1	5.0	6.0	7.0	8.0	9.0
2	8.0	9.0	10.0	11.0	2	10.0	11.0	12.0	13.0	14.0
					3	15.0	16.0	17.0	18.0	19.0

→ suma = frame1.add(frame2, fill_value=0)

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	4.0
1	9.0	11.0	13.0	15.0	9.0
2	18.0	20.0	22.0	24.0	14.0
3	15.0	16.0	17.0	18.0	19.0

Aplicando Lógica a Medida: `ufuncs` y `.apply`

Exploraremos cómo aplicar funciones de NumPy y lógica personalizada de manera eficiente en Pandas.

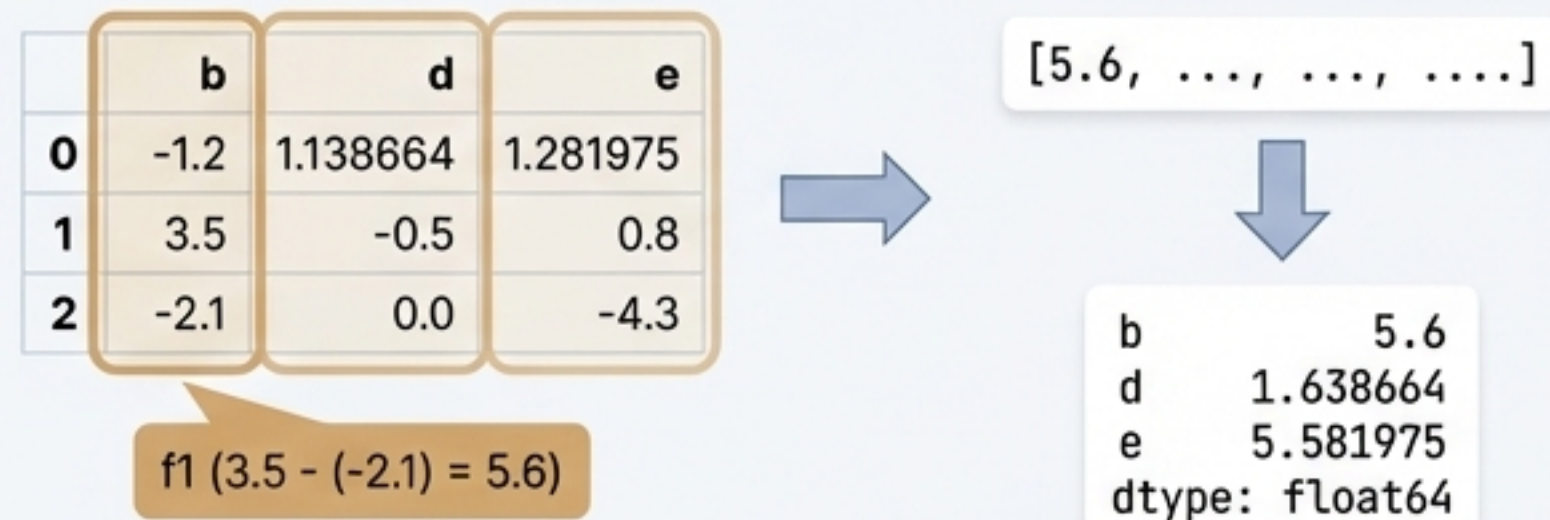
Sección 1: Funciones Universales (ufuncs) de NumPy

Las funciones de NumPy como `np.abs()` o `np.sqrt()` operan elemento por elemento en los objetos de Pandas de forma natural y vectorizada.

Antes				Después			
	b	d	e		b	d	e
0	-1.2	1.138664	1.281975	$\xrightarrow{\text{np.abs(frame)}}$	0	1.2	1.138664
1	3.5	-0.5	0.8		1	3.5	0.5
2	-2.1	0.0	-4.3		2	2.1	4.3

Aplicación por Columnas (por defecto)

```
frame.apply(f1)
```



Sección 2: El Poder de `.apply()`

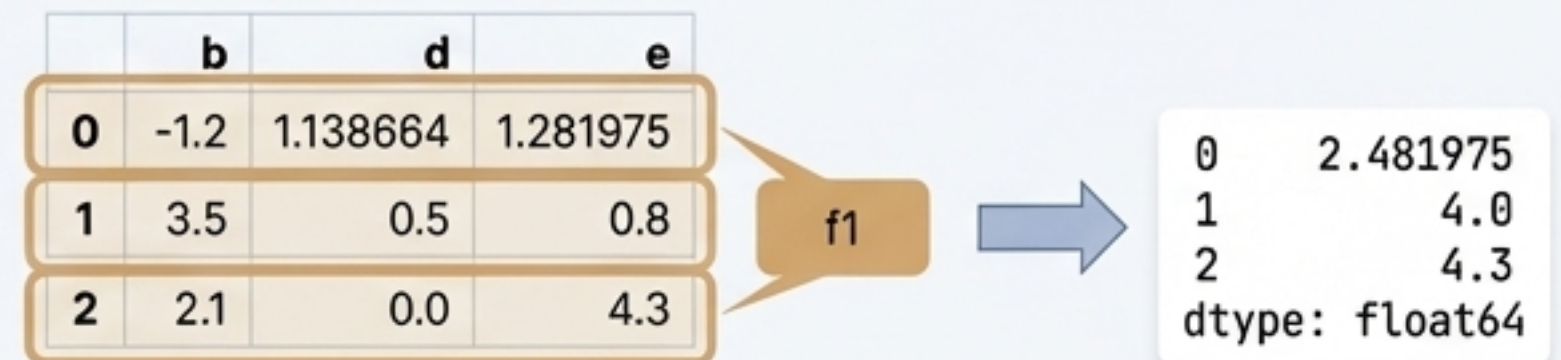
El método `.apply()` nos permite aplicar una función a lo largo de un eje del DataFrame (por columnas o por filas). Es la navaja suiza para operaciones personalizadas.

```
def f1(x):  
    return x.max() - x.min()
```

Calcula el rango
(diferencia entre máx. y mín.).

Aplicación por Filas

```
frame.apply(f1, axis="columns")
```



Imponiendo Orden: `sort\_index` vs. `sort\_values`

Antes

	d	a	b	c
Trenes	-0.697110	1.750307	-0.594645	0.685423
Barcos	0.410974	-0.973952	-1.070031	1.308861
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Coche	-0.505677	0.353713	0.625704	2.598641

1. Ordenar por Etiquetas con `sort\_index()`

Organiza los datos lexicográficamente según las etiquetas del índice o de las columnas.

Ejemplo 1: Ordenando Filas

```
frame.sort_index()
```

	d	a	b	c
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641
Trenes	-0.697110	1.750307	-0.594645	0.685423

Ejemplo 2: Ordenando Columnas

```
frame.sort_index(axis=1)
```

	a	b	c	d
Trenes	1.750307	-0.594645	0.685423	-0.697110
Barcos	-0.973952	-1.070031	1.308861	0.410974
Autobus	-1.622006	-1.249022	-0.926627	1.039543
Coche	0.353713	0.625704	2.598641	-0.505677

2. Ordenar por Valores con `sort\_values()`

Organiza los datos basándose en los valores de una o más columnas.

Ejemplo: Ordenar por múltiples columnas

```
frame.sort_values(["a", "c"])
```

Ordenar por la columna 'a', y luego por la 'c' para desempates.

	d	a ▲	b	c ▲
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Trenes	-0.697110	1.750307	-0.594645	0.685423
Coche	-0.505677	0.353713	0.625704	2.598641

Entendiendo la Jerarquía: Asignación de Rangos con `rank`

`rank()` asigna un rango a cada valor de una serie, de menor a mayor. Su característica más importante es cómo gestiona los empates, usando un método configurable.

Ejemplo con Series

```
obj = pd.Series([7, -5, 7, 4, 2, 0, 4])
```

0	7	
1	-5	
2	7	
3	4	
4	2	
5	0	
6	4	
dtype: int64		

Rango con Método `average` (por defecto)

```
obj.rank()
```

0	6.5
1	1.0
2	6.5
3	4.5
4	3.0
5	2.0
6	4.5

dtype: float64

Los dos valores `7` ocuparían los rangos 6 y 7. El promedio es $(6+7)/2 = 6.5$. De manera similar para los valores `4`, los rangos 4 y 5 promedian a 4.5.

Rango con Método `first`

```
obj.rank(method="first")
```

0	6.0
1	1.0
2	7.0
3	4.0
4	3.0
5	2.0
6	5.0

dtype: float64

Los empates se resuelven según el orden de aparición. El primer `7` obtiene el rango 6, el segundo obtiene el 7. Igualmente para los `4` (rangos 4 y 5).

Método	Descripción
"average"	Método por defecto. Asigna el rango medio a cada entrada de los valores empatados.
"min"	Utiliza el rango mínimo para el grupo empatado.
"max"	Utiliza el rango máximo para el grupo empatado.
"first"	Asigna rangos en el orden en que aparecen los valores en los datos.
"dense"	Igual que "min", pero el rango siempre aumenta en 1 entre grupos.

Destilando el Conocimiento: Estadísticas Descriptivas

La mayoría de los métodos estadísticos en Pandas son “reducciones”: toman un conjunto de valores (una columna o fila) y devuelven un único valor de resumen (ej. la suma, la media).

La Característica que Define a Pandas: Por defecto, los valores `NaN` se ignoran en estos cálculos. Esto es inmensamente práctico y evita que un solo valor faltante invalide todo un cálculo.

Datos con `NaN`

	uno	dos
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

Suma (ignorando `NaN`)

```
frame.sum(axis="index")
```

uno	9.25	} Los `NaN` son tratados como 0.
dos	-5.80	

Suma (sin ignorar `NaN`)

```
frame.sum(axis="index", skipna=False)
```

uno	NaN	} La presencia de un `NaN` invalida la suma.
dos	NaN	

Lista de Métodos Esenciales

Σ	SUM Suma de valores	μ	mean Media aritmética
	median Mediana (valor central)	σ	std Desviación estándar
#	count Número de valores no nulos	↓	min Valor mínimo
↑	max Valor máximo		idxmax Índice del valor máximo
	idxmin Índice del valor mínimo		

Dos Métodos para un Vistazo Rápido: `.describe()` y `value_counts()`

1. La Panorámica Numérica con `.describe()`

`.describe()` calcula un conjunto de estadísticas de resumen para las columnas numéricas de un DataFrame de una sola vez. Es el primer paso en casi cualquier análisis.

Age	Income	Score
25	50000	85
30	60000	92
30	45000	78
22	45000	88
28	55000	88

→ `.describe()`

	Age	Income	Score
count	4.0	4.0	4.0
mean	26.25	23.25	29.25
std	3.59	3.57	3.59
min	22.0	15.0	15.0
25%	24.25	37.25	38.25
50%	26.5	60.0	86.0
75%	28.5	78.0	78.5
max	30.0	100.0	88.0

2. Frecuencias de Categorías con `value_counts()`

`value_counts()` es el método más rápido para contar la frecuencia de cada valor único en una Serie. Es esencial para entender variables categóricas.

Antes

```
serie = pd.Series([3, 1, 2, 3, 2, 4, 1, 3])
```

```
[3, 1, 2, 3, 2, 4, 1, 3]
```

Después

```
conteo = serie.value_counts()
```

3	3
1	2
2	2
4	1

```
Name: count, dtype: int64
```


Tu Kit de Herramientas Esenciales de Pandas

Hemos completado nuestro viaje desde los datos en bruto hasta el conocimiento. Cada etapa tiene sus herramientas clave.



1. Dar Forma

- `reindex`
- `drop`



2. Acceder

- `loc`
- `iloc`
- `[]` (filtrado)



3. Transformar

- `.add()`
- `.apply()`



4. Ordenar

- `sort_index`
- `sort_values`
- `rank`



5. Destilar

- `sum`
- `mean`
- `describe`
- `value_counts`

Más Allá de lo Esencial: Próximos Pasos en tu Dominio de Pandas

Has dominado el núcleo de la manipulación de datos. Estas herramientas son la base del 90% del trabajo de limpieza y preparación. El siguiente nivel de maestría implica combinar y agregar datos de formas más complejas.

